A11 — *Outils d'analyse de contrats Solidity : que détectent-ils vraiment ?*

Animation — Cryptographie & Technologies Blockchain

Mohamed LO Ahmadoul Khadim TOURE Amadou Tidiane KANE

Séance S5 — vendredi 13 novembre 2026

École Supérieure Polytechnique, UCAD — Laboratoire LITA

Discutants : groupe 6 Rapporteurs : groupe 7

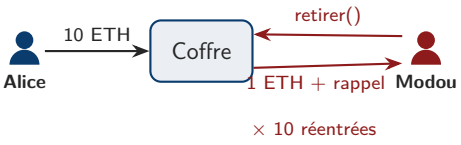


Ecole Supérieure Polytechnique
Laboratoire d'Informatique,
de Télécommunications et Applications
LITA

Contexte

Pourquoi ce sujet ?

- Un contrat intelligent est un **programme public, déterministe et immuable** : une faille déployée ne se corrige pas (Ch08).
- **The DAO** (2016) : une **réentrance** détourne environ 3,6 M d'ETH. La bifurcation dure qui suit fait naître Ethereum Classic **[atzei2017]**.
- Enjeu local : un diplôme de l'ESP vérifié sur une chaîne de blocs (sujet A13) reposerait lui aussi sur des contrats.



Question de recherche

Notre question (fiche officielle A11)

Les outils automatiques (Slither, Mythril, Echidna) détectent-ils les vulnérabilités réellement exploitées, et avec quel taux de faux positifs sur des contrats récents (Solidity 0.8) ?

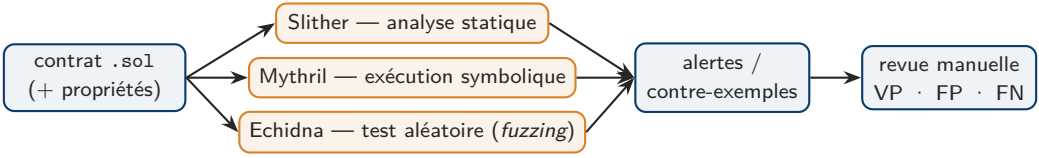
- **Hypothèse** : les outils sont complémentaires. Leur couverture dépend de la méthode et de la catégorie de faille. Les combiner l'améliore, sans garantir l'absence de faille.
- **Méthode envisagée** : au moins 20 contrats annotés (Solidity 0.8 en priorité), une matrice de confusion, puis précision, rappel et temps d'analyse, comparés à **[durieux2020]**.
- **Modèle d'attaque** : adversaire actif (Modou). Il déploie son propre contrat, appelle les fonctions publiques et exécute du code à chaque réception d'ETH, sans aucune clé privée d'autrui.

Mécanisme

Principe du mécanisme

Analyse automatique d'un contrat

Entrée : code Solidity ou bytecode, plus des propriétés pour Echidna. **Sortie** : des alertes (catégorie, emplacement, gravité) ou un contre-exemple. **Propriété visée** (réentrance) : l'état est mis à jour *avant* tout appel externe, et la somme des soldes reste au plus égale aux ETH détenus.



Exemple déroulé

Calcul pas à pas (mesuré sur le réseau Hardhat local)

1. **Paramètres** : Alice dépose 10 ETH, Modou mise 1 ETH, donc le coffre contient 11 ETH.
2. **Étape 1** : `retirer()` lit `montant = 1` et envoie 1 ETH *avant* `soldes[Modou] = 0`. La fonction `receive()` de Modou rappelle alors `retirer()`, et son solde enregistré vaut *toujours* 1.
3. **Étape 2** : chaque réentrée retire 1 ETH. Le coffre passe de 10 à 9, puis 8, et ainsi de suite jusqu'à 0, soit **10 réentrées**.
4. **Résultat et vérification** : Modou détient 11 ETH (1 + 10), et le coffre est vide. Pourtant, le registre indique encore 10 ETH pour Alice, qui ne peut plus retirer.

Ce qui peut mal tourner

Mauvais usage ou attaque connue

Réentrance : appel externe avant la mise à jour de l'état. Solidity 0.8 supprime les dépassements silencieux, *pas* la réentrance. **Impact** : perte ou gel de fonds, sans retour possible.

Taxonomie d'Atzei et al. [atzei2017]

- dépassement arithmétique (vérifié depuis 0.8, sauf `unchecked`) ;
- contrôle d'accès (Parity, 2017 : 513 774 ETH gelés) ;
- dépendance à l'horodatage ;
- exécution anticipée (*front-running*).

Du côté des outils

- fausse alerte : motif signalé mais non exploitable ;
- faille manquée : la logique métier leur échappe ;
- risques hors du code (oracles, ponts, clés), invisibles à l'analyse d'un fichier.

État de l'art

Travaux existants

Référence	Apport	Limite
Atzei et al., 2017 **[atzei2017]**	Taxonomie des attaques sur Ethereum (Solidity, EVM, chaîne de blocs)	Antérieure à Solidity 0.8 ; n'évalue aucun outil
Tsankov et al., 2018 **[tsankov2018]**	Securify : analyse statique par motifs de conformité et de violation	Limité aux propriétés encodées dans ses motifs
Feist et al., 2019 **[feist2019]**	Slither : analyse statique rapide sur une représentation intermédiaire (SlithIR)	Signale des motifs sans prouver leur exploitabilité
Durieux et al., 2020 **[durieux2020]**	9 outils, 69 contrats annotés : seulement 42 % des failles détectées par l'ensemble des outils ; 97 % d'environ 47 500 contrats réels signalés	Contrats antérieurs à Solidity 0.8, d'où un biais face aux contrats récents
Grieco et al., 2020 **[grieco2020]**	Echidna : test aléatoire guidé par des propriétés	Ne trouve que ce que décrivent les propriétés

5 sources primaires, dont les 3 lectures obligatoires (Atzei, Slither, Durieux).

Positionnement de notre projet

- **Ce qui n'a pas été étudié** : l'étude de référence **[durieux2020]** porte sur des contrats antérieurs à Solidity 0.8. On ignore si ses taux de détection et de faux positifs tiennent pour des contrats récents, où l'arithmétique est vérifiée par défaut.
- **Notre contribution** : mesurer, sur au moins 20 contrats annotés en priorité Solidity 0.8, ce que Slither, Mythril et Echidna détectent, ratent ou signalent à tort, par catégorie de faille. La vérité terrain est relue par deux membres, et le dépôt est reproductible (scripts, versions et sorties brutes).

Démonstration

Démonstration

```
// VulnerableVault.sol (coffre du cours)
function retirer() external {
  uint montant = soldes[msg.sender];
  require(montant > 0, "rien a retirer");
  // (1) envoi AVANT la mise a jour
  (bool ok, ) =
      msg.sender.call{value: montant}("");
  require(ok);
  // (2) remise a zero trop tard
  soldes[msg.sender] = 0;
}
```

- **À observer** : sur le réseau Hardhat local, l'attaquant vide le coffre ; la version corrigée (vérifications–effets–interactions + `nonReentrant`) résiste ; puis Slither sur les deux coffres.
- **Résultat obtenu** (10/11/2026, 7 tests sur 7) : coffre vulnérable 11 → 0 ETH en 10 réentrées ; coffre corrigé : Alice garde ses 10 ETH.
- **Slither 0.11.6** : `reentrancy-eth` (impact élevé) sur (1)–(2) ; aucune alerte de réentrance sur le coffre corrigé.
- Vidéo de secours de 2 minutes : [lien à ajouter].

Conclusion

À retenir

À retenir

- La réentrance vient de l'**ordre des instructions** : il faut mettre à jour l'état *avant* tout appel externe. Solidity 0.8 ne la supprime pas.
- Chaque famille d'outils a un angle mort. Une alerte ne prouve pas que la faille est exploitable, et **une absence d'alerte ne prouve pas la sécurité**.
- Sur notre coffre, Slither a bien signalé la réentrance réelle. La réponse à la question officielle viendra du corpus d'au moins 20 contrats annotés (résultats à venir).

Questions de quiz

Trois questions à choix multiple, remises à J+2 pour la banque de questions du contrôle individuel :

1. Quelle modification suffit à empêcher la réentrance dans le coffre du cours ?
   a) compiler en Solidity 0.8 b) remettre le solde à zéro avant l'envoi des fonds
   c) remplacer `uint` par `uint256` d) augmenter la limite de gaz
2. Quel outil repose sur l'exécution symbolique ?
   a) Slither b) Echidna c) Mythril d) Hardhat
3. Comment calcule-t-on la précision d'un outil de détection ?
   a) VP / (VP + FN) b) VP / (VP + FP) c) VN / (VN + FP) d) (VP + VN) / total

Références

[atzei2017] N. Atzei, M. Bartoletti, T. Cimoli. *A Survey of Attacks on Ethereum Smart Contracts (SoK)*. POST 2017, LNCS 10204. https://doi.org/10.1007/978-3-662-54455-6_8

[tsankov2018] P. Tsankov et al. *Securify: Practical Security Analysis of Smart Contracts*. ACM CCS 2018. https://doi.org/10.1145/3243734.3243780

[feist2019] J. Feist, G. Grieco, A. Groce. *Slither: A Static Analysis Framework for Smart Contracts*. WETSEB 2019. https://doi.org/10.1109/WETSEB.2019.00008

[durieux2020] T. Durieux, J. F. Ferreira, R. Abreu, P. Cruz. *Empirical Review of Automated Analysis Tools on 47,587 Ethereum Smart Contracts*. ICSE 2020. https://arxiv.org/abs/1910.10601

[grieco2020] G. Grieco, W. Song, A. Cygan, J. Feist, A. Groce. *Echidna: Effective, Usable, and Fast Fuzzing for Smart Contracts*. ISSTA 2020. https://doi.org/10.1145/3395363.3404366

[smartbugs] SmartBugs Curated, contrats Solidity vulnérables annotés. https://github.com/smartbugs/smartbugs-curated

[mendy2026] G. Mendy. *Ch08 — Contrats intelligents, confidentialité et applications*. Cours DIC3, ESP-UCAD / LITA, 2026.